Inteligencia artificial para leer documentos

Tokens, probabilidades y la API de Gemini

Introducción a la Ciencia de Datos — UM, FCEE

La inteligencia artificial y el modelo

Inteligencia artificial

Inteligencia artificial (IA). Sistema computacional que realiza tareas que asociamos a la inteligencia humana — percibir, reconocer, traducir, decidir— a partir de patrones obtenidos de datos.
Modelo. Función que transforma una entrada en una salida:

$$\hat{y} = f(x;\, \theta)$$

- x: la entrada (una foto, un texto)
- $\hat{y}$: la salida (el texto del ticket)
- θ: los **parámetros**, números que fijan el comportamiento de f

Un modelo de lenguaje actual tiene miles de millones de parámetros.
Russell y Norvig (2021), capítulo 1.

Entrenamiento e inferencia

Entrenamiento. Proceso de ajustar los parámetros θ para que las salidas del modelo se parezcan a las salidas correctas en un conjunto grande de ejemplos.
Función de pérdida. Número que mide cuánto se equivoca el modelo en los ejemplos. Entrenar es buscar los θ que la hacen chica.
Inferencia. Uso del modelo ya entrenado, con los parámetros fijos, para producir una salida ante una entrada nueva.
Hoy solo hacemos inferencia: usamos un modelo que Google entrenó. Cómo se entrena se estudia en Aprendizaje Automático.
Goodfellow, Bengio y Courville (2016), capítulo 5.

Red neuronal

Red neuronal. Modelo formado por **capas**. Cada capa recibe un vector de números, lo multiplica por una matriz de parámetros, le suma un vector y aplica una función no lineal:

$$h^{(\ell+1)} = \sigma\left(W^{(\ell)} h^{(\ell)} + b^{(\ell)}\right)$$

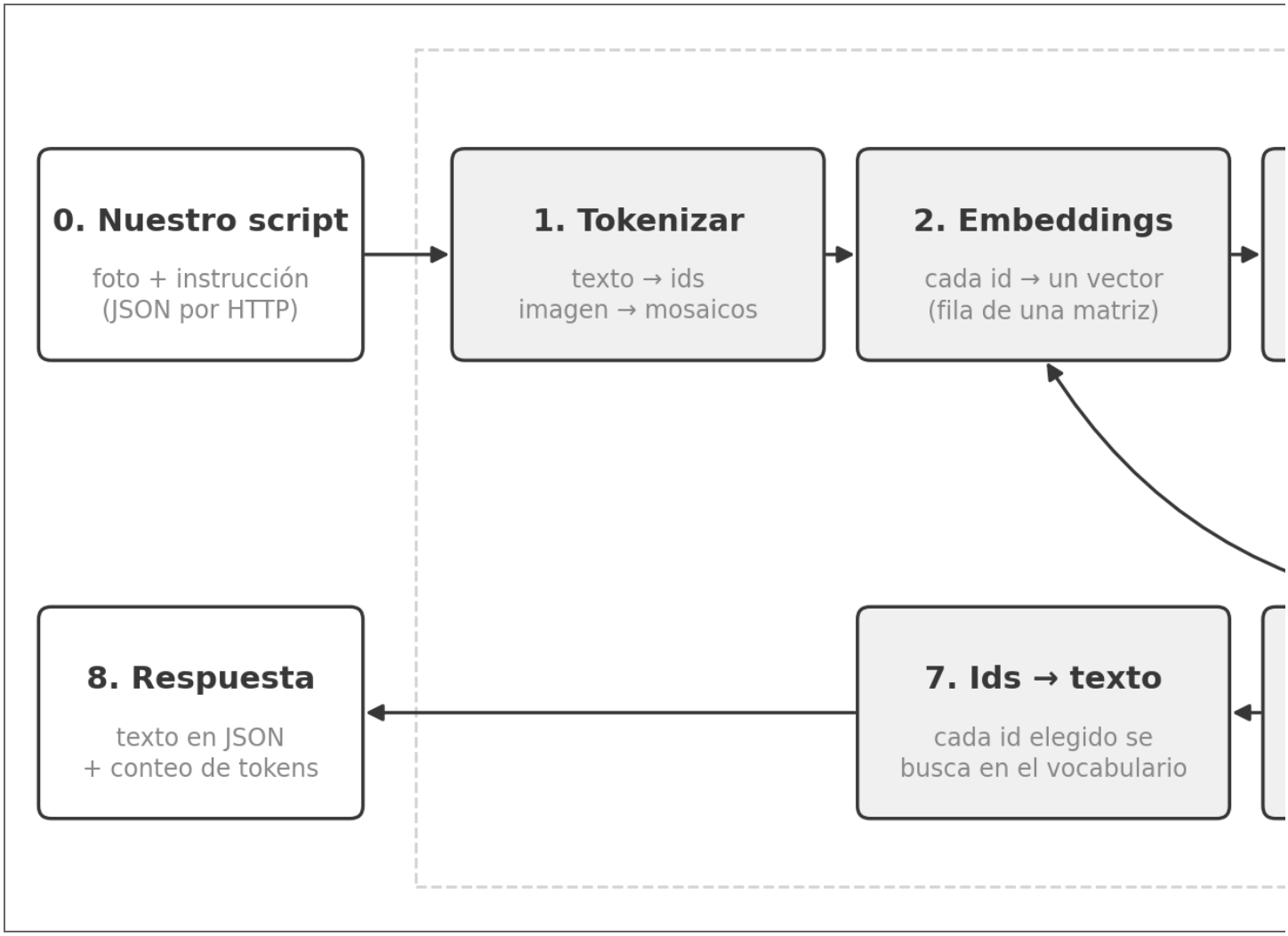
- $W^{(\ell)}, b^{(\ell)}$: parámetros de la capa ℓ
- σ: función no lineal, por ejemplo $\sigma(u) = \max(0, u)$

Apilando muchas capas, la composición de funciones simples puede representar relaciones muy complejas: por ejemplo, de píxeles a letras.
Goodfellow, Bengio y Courville (2016), capítulo 6.

Transformer

Transformer. Tipo de red neuronal diseñada para procesar **secuencias**. Recibe una lista de vectores —uno por token— y devuelve otra lista del mismo largo.
Atención. Operación central del Transformer: para calcular el nuevo vector de cada token, combina los vectores de los tokens anteriores, dándole más peso a los que son más relevantes.
Ejemplo: para decidir qué escribir después de «TOTAL $», la atención puede darle mucho peso a los tokens de la imagen donde está impreso el total.
Gemini está construido sobre Transformers de este tipo.
Vaswani et al. (2017). Gemini Team (2023), sección 2: los modelos Gemini se construyen sobre decodificadores Transformer.

El recorrido completo dentro de Gemini

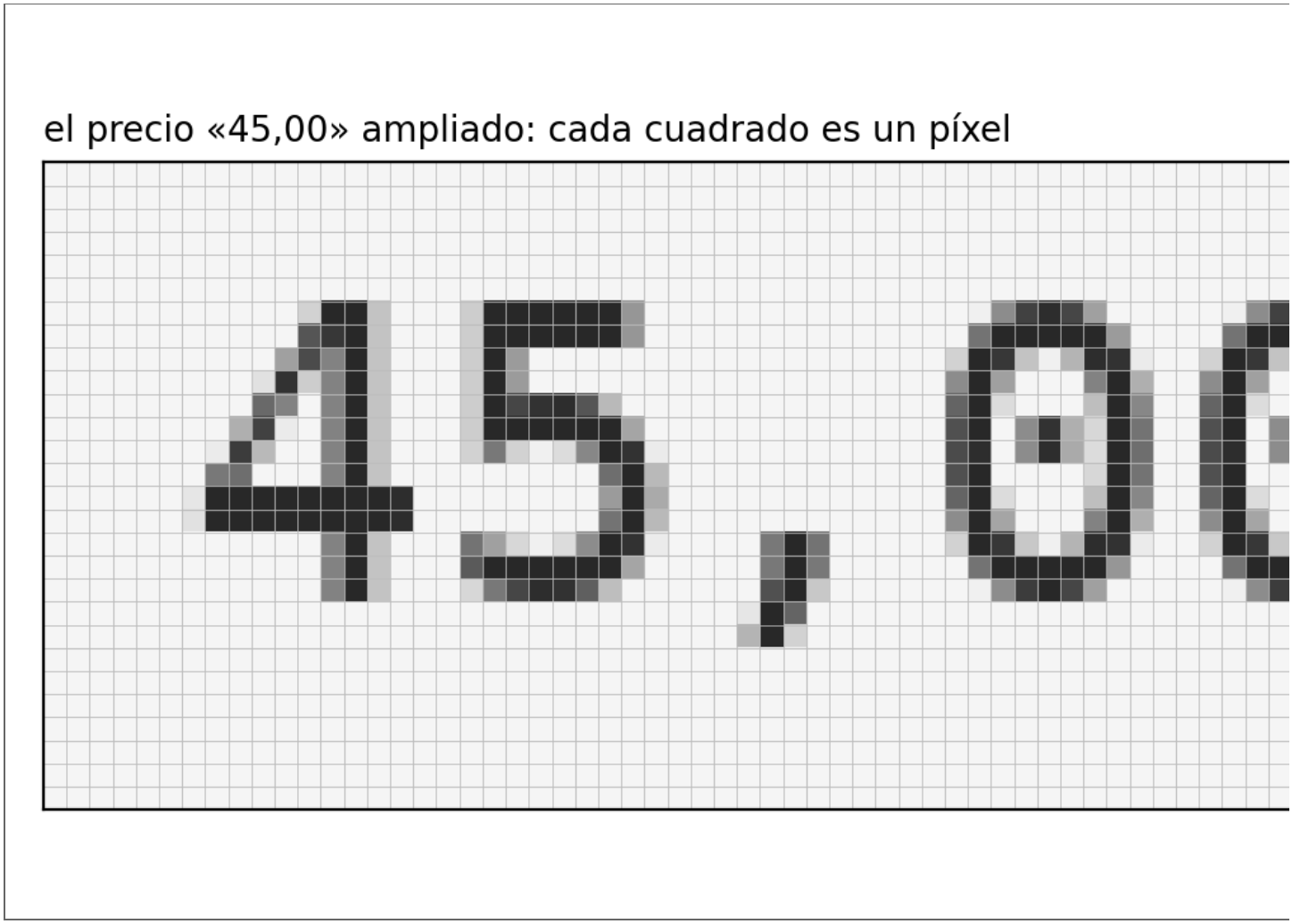


Nuestro script solo hace el paso 0 y recibe el paso 8. Los pasos 1 a 7 ocurren en el servidor de Google. Las diapositivas que siguen explican cada paso.
Esquema general de un modelo de lenguaje Transformer: Jurafsky y Martin, *Speech and Language Processing*. Aplicación a Gemini: Gemini Team (2023).

La entrada: imágenes y texto como números

Imagen digital

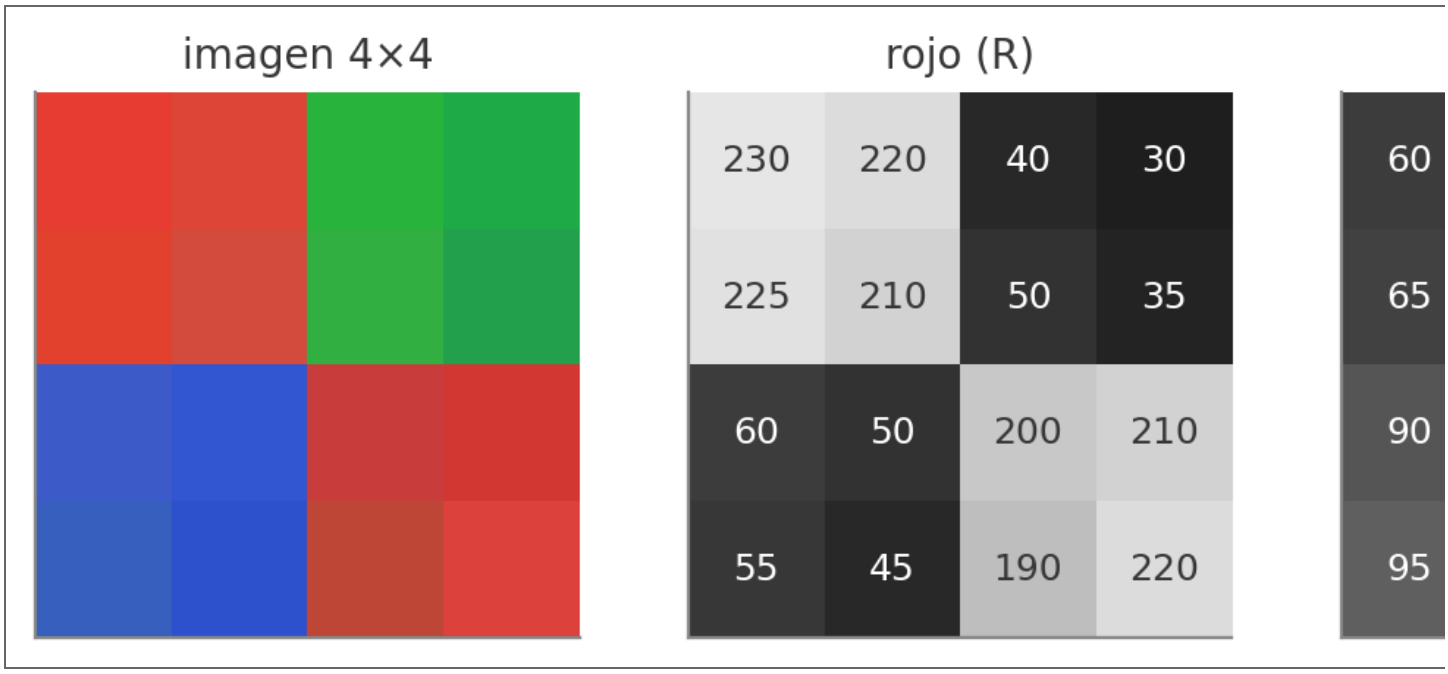
Imagen digital. Matriz de números. Cada entrada es un **píxel** y su valor es la intensidad de luz: en escala de grises, de 0 (negro) a 255 (blanco).



Gonzalez y Woods (2018), capítulo 2.

Imágenes en color

Imagen en color (RGB). Tres matrices del mismo tamaño, una por canal: rojo, verde y azul. Una foto de alto H y ancho W es un arreglo de $H \times W \times 3$ números.



Una foto de celular de 3000 × 4000 píxeles son 36 millones de números.
Gonzalez y Woods (2018), capítulo 6.

Token

Token. Unidad mínima en la que un modelo de lenguaje parte su entrada y su salida. Puede ser una letra, un fragmento de palabra o una palabra entera.
Tokenización. Procedimiento que corta un texto en tokens. Siempre corta igual: el mismo texto da siempre los mismos tokens.



El símbolo ␣ representa un espacio: el espacio forma parte del token. Los **números se cortan dígito por dígito**: `109` son tres tokens, `1`, `0` y `9`.
Segmentación ilustrativa. Dígitos separados: Gemma Team (2024), sección sobre el tokenizador. En Gemini, un token equivale en promedio a unos 4 caracteres: Google, documentación de la API de Gemini, «Tokens».

Vocabulario

Vocabulario. Lista fija y finita de todos los tokens que el modelo puede leer o escribir.

- Se construye **antes** de entrenar el modelo, con un algoritmo aplicado a una colección enorme de textos (el **corpus**).
- Una vez construido **no cambia**: el modelo se entrena con ese vocabulario y queda atado a él.
- El programa que corta el texto usando el vocabulario se llama **tokenizador**. Es parte del modelo, no de nuestro código.

  Gemini usa un tokenizador llamado **SentencePiece**. Google no publica el vocabulario de Gemini, pero sí el de **Gemma**, un modelo abierto que usa parte del mismo tokenizador: tiene 256.000 tokens.
  Gemini Team (2023), sección 4. Gemma Team (2024), sección sobre el tokenizador. Kudo y Richardson (2018).

Cómo se construye el vocabulario: BPE

BPE (*byte pair encoding*, codificación por pares). Algoritmo que arma el vocabulario uniendo, paso a paso, los pares de tokens vecinos más frecuentes en el corpus.

1. Empezar con un vocabulario de símbolos sueltos: letras, dígitos, signos y el espacio.
2. Partir cada palabra del corpus en esos símbolos.
3. Contar cuántas veces aparece cada par de tokens vecinos.
4. Unir el par más frecuente en un token nuevo y agregarlo al vocabulario.
5. Repetir 3 y 4 hasta llegar al tamaño deseado (por ejemplo, 256.000 tokens).

   Cada unión agrega exactamente un token. Las secuencias frecuentes terminan siendo un solo token; las raras quedan partidas en pedazos.
   Sennrich, Haddow y Birch (2016). Jurafsky y Martin, *Speech and Language Processing*, capítulo sobre palabras y tokens. SentencePiece implementa BPE y una variante: Kudo y Richardson (2018).

BPE con un ejemplo

Corpus de juguete:

palabra	veces
de	12
leche	10
dulce	6
del	5
lechuga	4
queso	3

Vocabulario inicial: 12 símbolos, `a c d e g h l o q s u ␣`

paso	par más frecuente	veces	token nuevo	`␣leche` queda	`␣lechuga` queda
1	␣ + d	23	␣d	␣·l·e·c·h·e	␣·l·e·c·h·u·g·a
2	␣d + e	17	␣de	␣·l·e·c·h·e	␣·l·e·c·h·u·g·a
3	␣ + l	14	␣l	␣l·e·c·h·e	␣l·e·c·h·u·g·a
4	␣l + e	14	␣le	␣le·c·h·e	␣le·c·h·u·g·a
5	␣le + c	14	␣lec	␣lec·h·e	␣lec·h·u·g·a

paso	par más frecuente	veces	token nuevo	␣leche queda	␣lechuga queda
6	␣lec + h	14	␣lech	␣lech·e	␣lech·u·g·a
7	␣lech + e	10	␣leche	␣leche	␣lech·u·g·a
8	␣d + u	6	␣du	␣leche	␣lech·u·g·a

El punto · separa tokens. «veces» del paso 1: ␣d aparece al inicio de *de* (12), *dulce* (6) y *del* (5): 12 + 6 + 5 = 23.

Cálculo propio con el algoritmo de Sennrich et al. (2016) (archivo `bpe_ejemplo.py`).

Resultado: palabras frecuentes enteras, raras partidas

Después de 8 pasos el vocabulario tiene 12 + 8 = 20 tokens, y:

palabra	frecuencia en el corpus	tokens	cantidad
leche	10	`␣leche`	1
lechuga	4	`␣lech u g a`	4
dulce	6	`␣du l c e`	4

Lo mismo pasa a gran escala. El vocabulario distingue mayúsculas de minúsculas, y en los textos «leche» aparece mucho más que «LECHE». Por eso un ticket, escrito en mayúsculas, probablemente se parte en más tokens que el mismo texto en minúsculas.

texto	tokens (ilustrativo)
`dulce de leche`	`dulce␣de␣leche` → 3
`DULCE DE LECHE`	`DUL CE␣DE␣LE CHE` → 5

La cantidad real de tokens de un texto se puede consultar con la función `countTokens` de la API de Gemini.

De dónde salen los ids

Id (identificador). Número de la posición del token en la lista del vocabulario. Es solo un número de orden, como el número de fila de una tabla.

El vocabulario del ejemplo, en el orden en que se construyó:

id	token	id	token
0	a	10	u
1	c	11	␣
2	d	12	␣d
3	e	13	␣de
4	g	14	␣l
5	h	15	␣le
6	l	16	␣lec
7	o	17	␣lech
8	q	18	␣leche
9	s	19	␣du

Tokenizar «dulce de leche»:

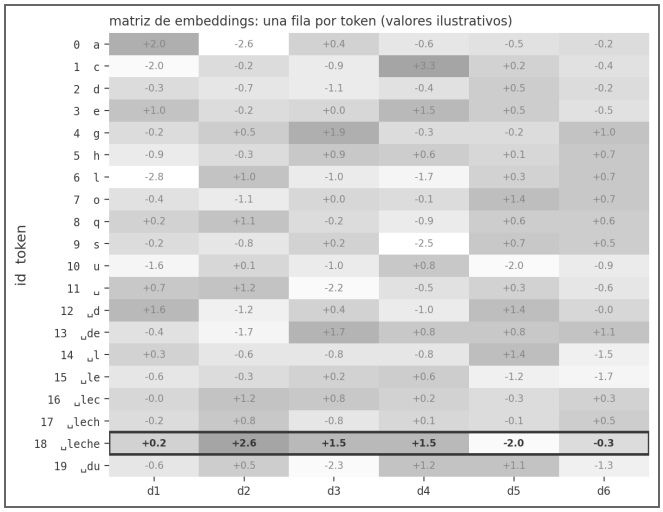
token	␣du	l	c	e	␣de	␣leche
id	19	6	1	3	13	18

El texto entra al modelo como la lista de números

$$[19,\ 6,\ 1,\ 3,\ 13,\ 18]$$

El modelo **nunca ve letras**: solo estos números. En Gemma los ids van de 0 a 255.999.

Del id al embedding



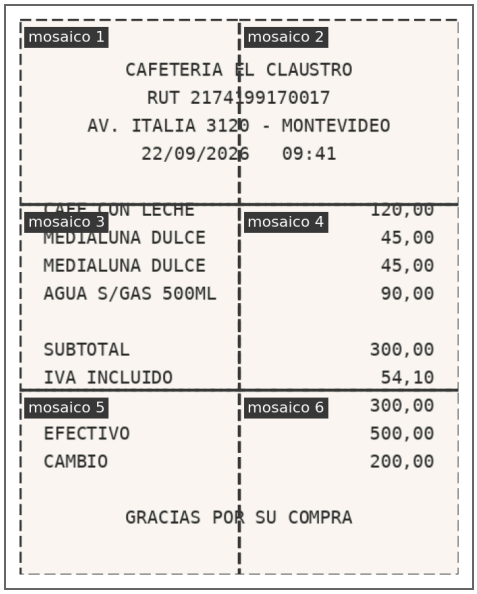
Embedding (representación vectorial). Vector de números reales asociado a cada token.
Matriz de embeddings. Matriz con **una fila por token del vocabulario**. La fila número i es el embedding del token con id i.
El id sirve para buscar la fila: el id 18 dice «usá la fila 18», que es el vector de `␣leche`.
Los valores de la matriz son parámetros: se ajustan

durante el entrenamiento.

En un modelo real cada fila tiene miles de números.

Valores ilustrativos. Jurafsky y Martin, *Speech and Language Processing*, capítulos sobre embeddings y Transformers.

La imagen también se convierte en tokens



Modelo multimodal. Modelo que recibe más de un tipo de entrada —texto, imagen, audio— y los procesa juntos como una sola secuencia.
La imagen no pasa por el vocabulario de palabras: se corta en pedazos y cada pedazo se convierte directamente en vectores, que ocupan lugares en la secuencia igual que los tokens de texto.
En Gemini:

- una imagen de hasta 384 × 384 píxeles cuenta **258 tokens**
- una imagen más grande se corta en **mosaicos** de 768 × 768 píxeles, y

cada mosaico cuenta 258 tokens

Esquema ilustrativo. Conteo de tokens: Google, documentación de la API de Gemini, «Image understanding» y «Tokens». Entrada de texto e imágenes intercaladas: Gemini Team (2023), sección 2.

La salida: probabilidades

Modelo de lenguaje

Modelo de lenguaje. Modelo que, dada una secuencia de tokens, asigna una probabilidad a cada token posible del vocabulario como **siguiente** token:

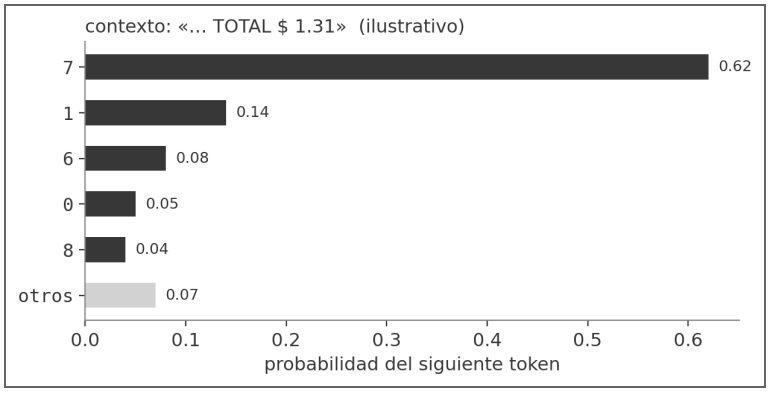
$$P(t_{n+1} = v \mid t_1, t_2, \ldots, t_n) \quad \text{para cada } v \text{ del vocabul}$$

La probabilidad de un texto completo se obtiene encadenando:

$$P(t_1, \ldots, t_n) = \prod_{i=1}^{n} P(t_i \mid t_1, \ldots, t_{i-1})$$

En nuestro caso, los primeros tokens son los de la imagen y los de la instrucción; los siguientes son los que el modelo va escribiendo.
Jurafsky y Martin, *Speech and Language Processing*, capítulo sobre modelos de lenguaje con n-gramas.

Siguiente token



Después de leer la imagen y de haber escrito «TOTAL $ 1.31», el modelo tiene una distribución de probabilidad sobre lo que sigue.
Si la imagen es nítida, casi toda la probabilidad cae en el token correcto.
Si está borrosa, la probabilidad se reparte entre candidatos parecidos.
Valores ilustrativos.

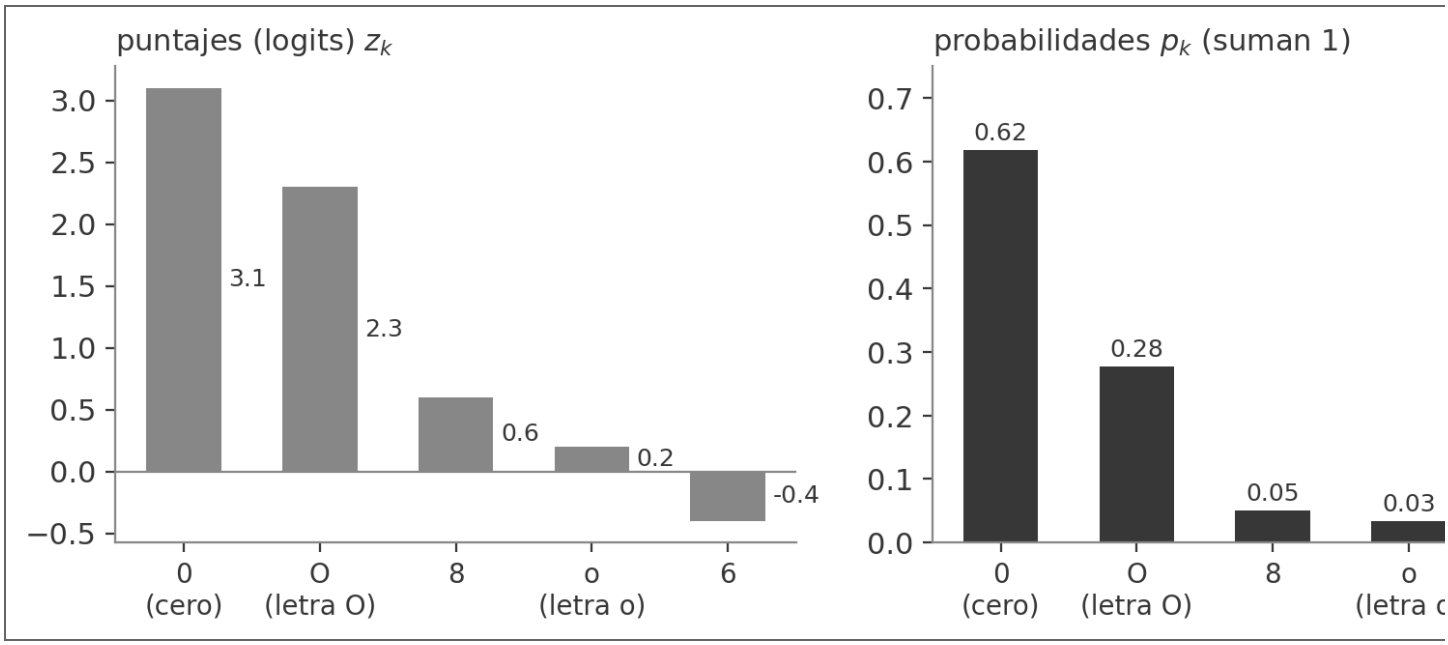
Jurafsky y Martin, *Speech and Language Processing*, capítulo sobre modelos de lenguaje con Transformers.

De puntajes a probabilidades: softmax

La última capa de la red da un **puntaje** (logit) z_k por cada token k del vocabulario. La función **softmax** los convierte en probabilidades:

$$p_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$$

Cada p_k está entre 0 y 1, y todas suman 1. El puntaje más alto recibe la probabilidad más alta.



Goodfellow, Bengio y Courville (2016), sección 6.2.2.3.

Softmax paso a paso

Como los dígitos son tokens separados, el modelo escribe «109» dígito por dígito. Puntajes para el token que sigue a «1» (valores ilustrativos):

candidato	z_k	e^{z_k}	$p_k = e^{z_k}/35{,}88$
0 (cero)	3,1	22,20	**0,62**
O (letra)	2,3	9,97	0,28
8	0,6	1,82	0,05
o (letra)	0,2	1,22	0,03
6	−0,4	0,67	0,02
suma		**35,88**	**1,00**

El cero y la letra O se parecen en la imagen: por eso la letra se lleva el 28 %. Así se producen las confusiones como leer `109,00`, con la letra O en lugar del cero.

Del puntaje al texto: los ids a la salida

La última capa da **un puntaje por cada id del vocabulario**: en Gemma, 256.000 puntajes. El softmax los convierte en 256.000 probabilidades que suman 1. Con el vocabulario de juguete (20 ids), después de leer `␣du l c e ␣de`:

id	token	probabilidad
18	`␣leche`	**0,81**
17	`␣lech`	0,07
13	`␣de`	0,03
…	(los otros 17 ids)	0,09

El modelo elige un **id** (el 18), lo busca en la lista del vocabulario y recién ahí aparece el texto: `␣leche`. Este paso final, de ids a texto, se llama **detokenización**.
Probabilidades ilustrativas. Jurafsky y Martin, *Speech and Language Processing*, capítulo sobre modelos de lenguaje con Transformers.

Generación autorregresiva

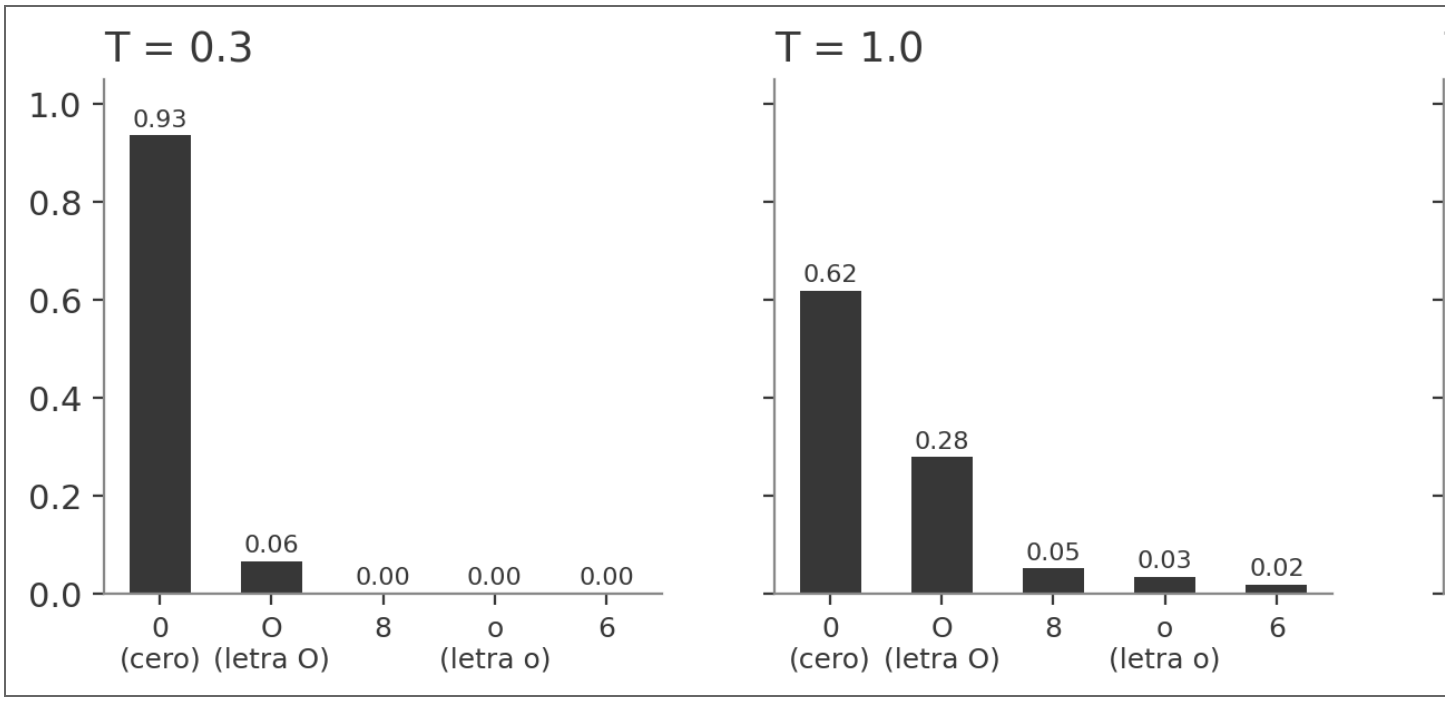
Generación autorregresiva. El modelo escribe la respuesta de a un token por vez: calcula la distribución del siguiente token, elige uno, lo agrega a la secuencia y repite con la secuencia más larga.

paso	lo que ya está escrito	token elegido
1	*(imagen + instrucción)*	`DUL`
2	`DUL`	`CE`
3	`DULCE`	`␣DE`
4	`DULCE DE`	`␣LE`
…	…	…
13	`DULCE DE LECHE 400G $`	`␣`
14	`DULCE DE LECHE 400G $`	`1`
15	`DULCE DE LECHE 400G $ 1`	`0`

Termina cuando elige un token especial de **fin de secuencia** o llega al máximo de tokens permitido.
Jurafsky y Martin, *Speech and Language Processing*, capítulo sobre modelos de lenguaje con Transformers.

Cómo se elige el token: temperatura

Decodificación voraz. Elegir siempre el token de mayor probabilidad.
Muestreo. Sortear el token según las probabilidades p_k.
Temperatura T. Parámetro que divide los puntajes antes del softmax: $p_k \propto e^{z_k/T}$.



Con T chica la distribución se concentra en el más probable. Para leer un ticket conviene temperatura baja: queremos la lectura más probable, no variedad.
Jurafsky y Martin, *Speech and Language Processing*, sección sobre muestreo. El parámetro `temperature` de

Gemini: Google, documentación de la API, «generateContent».

Alucinación

Alucinación. Salida del modelo que es plausible pero no está respaldada por la entrada.
El modelo elige tokens probables **dado todo lo que aprendió**, no solo dada la imagen. En un ticket:

- completar un precio borroso con un valor típico
- «corregir» un total para que coincida con la suma
- agregar un ítem que suele aparecer en tickets parecidos
- escribir una fecha con formato distinto al impreso

Un OCR clásico, cuando no puede leer, deja un hueco o un carácter raro. Un modelo de lenguaje tiende a escribir algo que **parece** correcto. Por eso hay que controlar la salida.
Ji et al. (2023).

Lo que se sabe de Gemini y lo que no

Publicado por Google:

- se construye sobre **decodificadores Transformer** (Gemini Team, 2023)
- usa el tokenizador **SentencePiece**, entrenado sobre una muestra grande del corpus (Gemini Team, 2023)
- recibe texto intercalado con imágenes, audio y video (Gemini Team, 2023)
- un token ≈ 4 caracteres; una imagen chica = 258 tokens (documentación de la API)

No publicado:

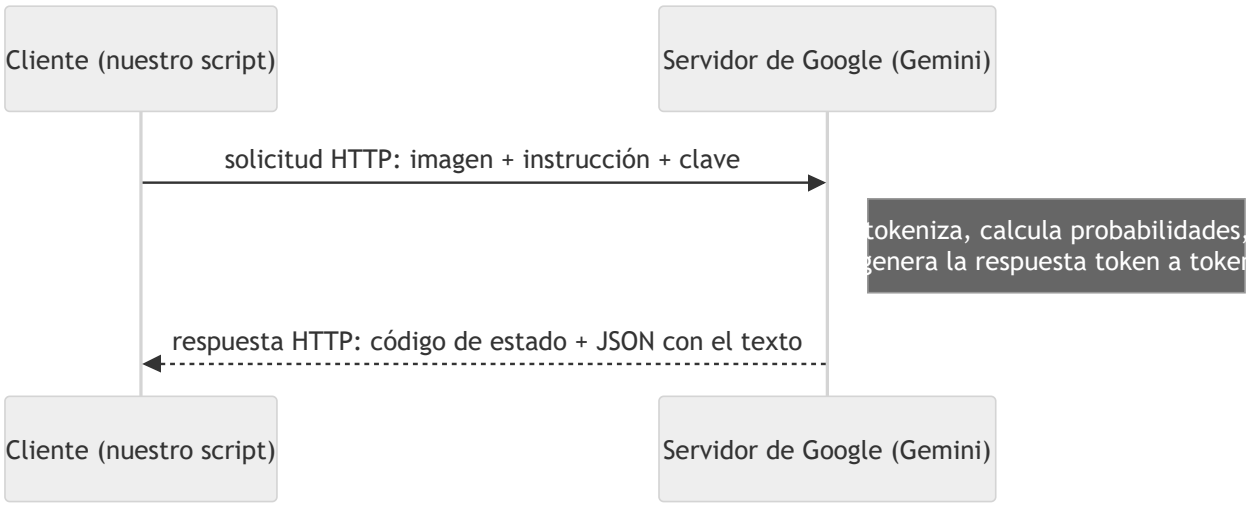
- el vocabulario exacto y sus ids
- la cantidad de parámetros y los detalles de las versiones actuales
- con qué textos e imágenes exactos se entrenó

Por eso, en esta clase, los ids y las probabilidades concretas son **ilustrativos**: el mecanismo es el descripto, los números no son los de Gemini.

La API

API

API (interfaz de programación de aplicaciones). Conjunto de reglas que permite a un programa pedirle un servicio a otro programa.
Cliente. Programa que hace el pedido: nuestro script.
Servidor. Computadora que recibe el pedido, lo procesa y responde: la de Google, donde está el modelo.



HTTP

HTTP. Protocolo con el que se comunican cliente y servidor en la web. Cada intercambio es una **solicitud** y una **respuesta**.
Método. Tipo de pedido. `GET` pide un recurso; `POST` envía datos para que el servidor los procese. Para mandar una imagen se usa `POST`.
Encabezados (headers). Pares `nombre: valor` con información sobre la solicitud: qué formato tiene el cuerpo, quién la hace.
Cuerpo (body). Los datos que se envían: en nuestro caso, un JSON con la imagen y la instrucción.
MDN Web Docs, «An overview of HTTP».

Anatomía de la dirección (endpoint)

Endpoint. Dirección a la que se envía la solicitud.
`https://generativelanguage.googleapis.com/v1beta/models/g`
`3.8-flash:generateContent`

- **esquema** `https`: HTTP cifrado
- **servidor**: la máquina de Google que atiende la API de Gemini
- **versión** `v1beta`: la API cambia; la versión fija qué reglas se usan
- **recurso**: qué modelo se usa
- **acción** `generateContent`: generar una respuesta

Google, documentación de la API de Gemini, «generateContent».

La solicitud completa

```
POST /v1beta/models/gemini-3.8-flash:generateContent HTTP/1.1
Host: generativelanguage.googleapis.com
Content-Type: application/json
x-goog-api-key: AIza...clave...

{
  "contents": [{
    "parts": [
      {"text": "Transcribí los ítems de este ticket como JSON ..."},
      {"inline_data": {"mime_type": "image/jpeg",
                       "data": "/9j/4AAQSkZJRgABAQ..."}}
    ]
  }],
  "generationConfig": {"temperature": 0,
                       "responseMimeType": "application/json"}
}
```

`contents` → `parts`: la entrada es una lista de partes, texto e imagen. `generationConfig` fija la temperatura y pide la respuesta en JSON.

Google, documentación de la API de Gemini, «generateContent» e «Image understanding».

Base64: cómo viaja la imagen dentro del JSON

Base64. Codificación que escribe cualquier secuencia de bytes usando solo 64 caracteres imprimibles (`A–Z`, `a–z`, `0–9`, `+`, `/`). Cada 3 bytes se convierten en 4 caracteres.
Hace falta porque el JSON solo admite texto, y una imagen son bytes arbitrarios.

paso	resultado
bytes del archivo (hexadecimal)	`FF D8 FF`
los mismos 24 bits	`11111111 11011000 11111111`
en grupos de 6 bits	`111111 111101 100011 111111`
valor de cada grupo (0 a 63)	63, 61, 35, 63
carácter de la tabla base64	`/ 9 j /`

Todo JPEG empieza con los bytes `FF D8 FF`: por eso en base64 empieza con `/9j/`. El texto resultante ocupa alrededor de un tercio más que el archivo.
Josefsson (2006), RFC 4648, sección 4.

Clave de API y autenticación

Autenticación. Procedimiento con el que el servidor verifica quién hace la solicitud.
Clave de API. Cadena secreta que identifica el proyecto que hace la solicitud. Se envía en el encabezado `x-goog-api-key`.

- La clave se obtiene gratis en Google AI Studio con una cuenta de Google.
- Quien tiene la clave usa la cuota del dueño: no se comparte ni se sube a GitHub.
- En el código se lee de una **variable de entorno** o de un archivo excluido del repositorio.

  Google, documentación de la API de Gemini, «Using Gemini API keys».

La respuesta


```
{
  "candidates": [{
    "content": {
      "parts": [{"text": "{\"comercio\": \"CAFETERIA EL CLAUSTRO\", \"items\": [
    },
    "finishReason": "STOP"
  }],
  "usageMetadata": {
    "promptTokenCount": 1310,
    "candidatesTokenCount": 142,
    "totalTokenCount": 1452
  }
}
```


- `candidates[0].content.parts[0].text`: el texto generado
- `finishReason`: por qué terminó (`STOP` = fin de secuencia)
- `usageMetadata`: tokens de entrada (imagen + instrucción), de salida y total

  Cantidades de tokens ilustrativas.
  Google, documentación de la API de Gemini, «generateContent».

Códigos de estado

Código de estado. Número de tres cifras con el que el servidor indica cómo terminó la solicitud.

código	significado	qué hacer
200	todo bien	leer el JSON
400	solicitud mal armada	revisar el cuerpo: campos, base64
403	clave inválida o sin permiso	revisar la clave
429	demasiadas solicitudes	esperar y reintentar
500 / 503	error o sobrecarga del servidor	reintentar más tarde

Fielding, Nottingham y Reschke (2022), RFC 9110, sección 15. Google, documentación de la API de Gemini, «Troubleshooting».

Límites y costo

Cuota. Cantidad máxima de solicitudes o tokens permitida por minuto y por día.

- **Nivel gratuito**: sin costo y sin tarjeta, con cuotas por minuto y por día que dependen del modelo.
- **Nivel pago**: se cobra por token, con precios distintos para tokens de entrada y de salida.
- Tamaño máximo de una solicitud con la imagen incluida: 20 MB.
- En el nivel gratuito, Google usa el contenido enviado para mejorar sus productos. En el pago, no.

  Por eso una foto grande «cuesta» más que una chica: tiene más mosaicos, y por lo tanto más tokens.
  Google, documentación de la API de Gemini, «Pricing» y «Rate limits». Límite de 20 MB: «Image understanding».

El programa cliente en Python

```python
import base64, json, os, requests

URL = ("https://generativelanguage.googleapis.com/v1beta/"
       "models/gemini-3.8-flash:generateContent")
clave = os.environ["GEMINI_API_KEY"]

with open("ticket.jpg", "rb") as f:
    imagen = base64.b64encode(f.read()).decode("ascii")

cuerpo = {
    "contents": [{"parts": [
        {"text": INSTRUCCION},
        {"inline_data": {"mime_type": "image/jpeg", "data": imagen}},
    ]}],
    "generationConfig": {"temperature": 0, "responseMimeType": "application/json
}
r = requests.post(URL, headers={"x-goog-api-key": clave}, json=cuerpo, timeout=1
r.raise_for_status()                                # corta si el código no es 200
texto = r.json()["candidates"][0]["content"]["parts"][0]["text"]
ticket = json.loads(texto)                          # de texto JSON a diccionario
```

El programa cliente en R

```r
library(httr2); library(jsonlite)

clave  <- readLines("clave_gemini.txt", warn = FALSE)[1]
url    <- "https://generativelanguage.googleapis.com/v1beta/models/gemini-3.8-fl
bytes  <- readBin("ticket.jpg", what = "raw", n = file.size("ticket.jpg"))
imagen <- gsub("[\r\n]", "", base64_enc(bytes))          # foto -> base64

cuerpo <- list(contents = list(list(parts = list(
  list(text = instruccion),
  list(inline_data = list(mime_type = "image/jpeg", data = imagen))))),
  generationConfig = list(temperature = 0))

respuesta <- request(url) |>
  req_headers(`x-goog-api-key` = clave) |>                # la clave en un enc
  req_body_json(cuerpo) |>                                # cuerpo JSON, métod
  req_perform()

datos <- resp_body_json(respuesta)                        # JSON -> lista
texto <- datos$candidates[[1]]$content$parts[[1]]$text
```

Los mismos pasos que en Python: leer la foto, pasarla a base64, armar el JSON, enviar, leer la respuesta. Archivos completos: `leer_ticket.py` y `leer_ticket.R`.

Paquete httr2: Wickham (2024), documentación en httr2.r-lib.org. Google, documentación de la API de Gemini, «generateContent».

La instrucción (prompt)

Prompt. Texto que se le da al modelo junto con la entrada para indicar qué tarea hacer y en qué formato responder.

```
Transcribí este ticket de compra uruguayo. Respondé solo con JSON:
{"comercio": texto, "fecha": "dd/mm/aaaa",
 "items": [{"descripcion": texto, "monto": número}],
 "total": número}
- Copiá las descripciones como están impresas.
- Los montos usan punto de miles y coma decimal: 1.250,00 es 1250.
- Un descuento es un monto negativo.
- No incluyas SUBTOTAL, IVA, EFECTIVO, CAMBIO ni TARJETA como ítems.
- Si algo no se puede leer, poné null. No lo completes.
```

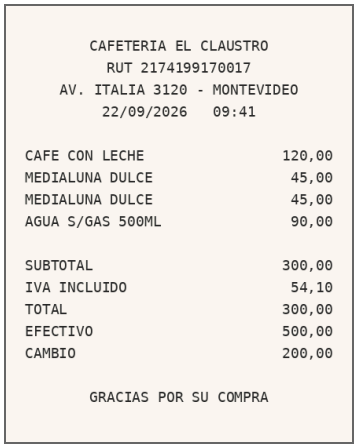
Cada regla del prompt corresponde a un error posible. La última apunta a la alucinación.

Google, documentación de la API de Gemini, «Prompt design strategies».

Del texto a la tabla

Dato no estructurado y dato estructurado

Dato no estructurado. Información sin filas ni columnas definidas: texto corrido, imágenes, audio.
Dato estructurado. Información organizada en una tabla: cada fila es una unidad de observación y cada columna una variable.



```
        CAFETERIA EL CLAUSTRO
          RUT 2174199170017
     AV. ITALIA 3120 - MONTEVIDEO
          22/09/2026   09:41

CAFE CON LECHE                120,00
MEDIALUNA DULCE                45,00
MEDIALUNA DULCE                45,00
AGUA S/GAS 500ML               90,00

SUBTOTAL                      300,00
IVA INCLUIDO                   54,10
TOTAL                         300,00
EFECTIVO                      500,00
CAMBIO                        200,00

        GRACIAS POR SU COMPRA
```

comercio	fecha	descripcion	monto
Cafetería El Claustro	22/09/2026	café con leche	120
Cafetería El Claustro	22/09/2026	medialuna dulce	45
Cafetería El Claustro	22/09/2026	medialuna dulce	45

comercio	fecha	descripcion	monto
Cafetería El Claustro	22/09/2026	agua s/gas 500ml	90

Unidad de observación: el ítem comprado.

Errores de lectura

Error de lectura. Diferencia entre lo que dice el documento y lo que devuelve el modelo.

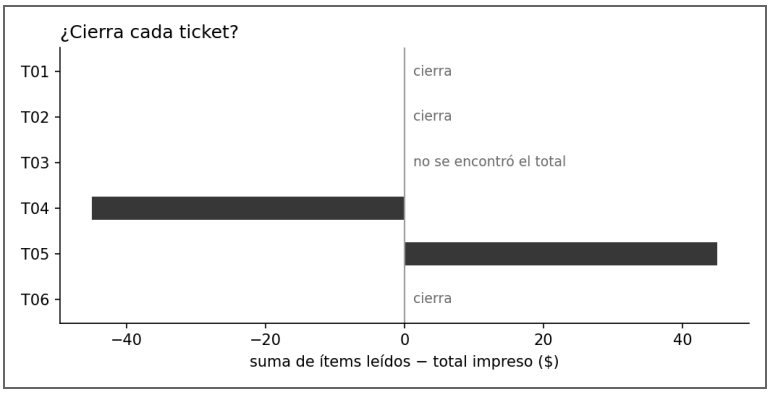
tipo	definición	ejemplo
sustitución	un carácter se lee como otro	`109,00` → `109,00` (letra O en lugar de cero)
omisión	algo impreso no aparece	renglón desteñido
inserción	aparece algo que no está	un ítem que no existe
error de estructura	se lee bien pero va a otro campo	la cantidad `2 x 45,00` como un ítem más
alucinación	valor plausible sin respaldo en la imagen	un precio borroso completado con un valor típico

Control de consistencia

Control de consistencia.
Verificación que usa una relación que los datos deben cumplir, sin necesidad de conocer el valor correcto.
En un ticket:

$$\sum_{\text{ítems}} \text{monto} = \text{TOTAL}$$

Si la diferencia no es cero, falta un ítem (T04) o sobra uno (T05).
Ojo: un modelo de lenguaje puede «acomodar» un número para que la suma cierre. Que cierre no garantiza que esté bien.


¿Cierra cada ticket?
T01
T02
T03
T04
T05
T06
cierra
cierra
no se encontró el total
cierra
−40
−20
0
20
40
suma de ítems leídos − total impreso ($)

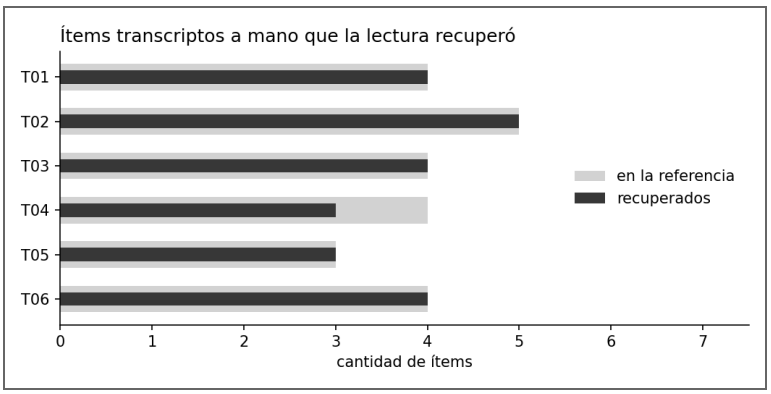
Referencia y tasa de recuperación

Referencia. Versión de los datos que se toma como correcta: la transcripción hecha a mano.

Tasa de recuperación. Proporción de los elementos de la referencia que aparecen en lo leído:

$$\text{tasa} = \frac{\text{ítems recuperados}}{\text{ítems en la referencia}}$$

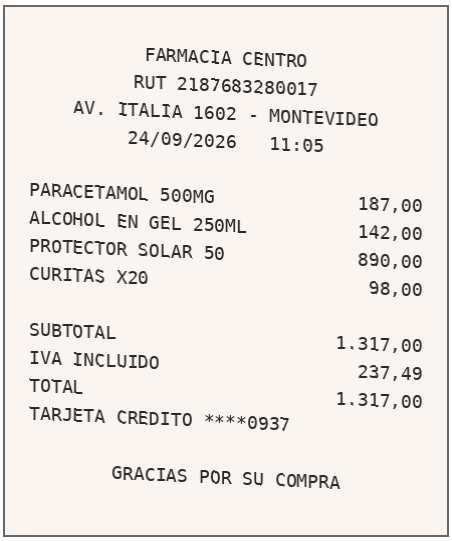
T04: $3/4 = 0{,}75$.



Datos personales

Dato personal y dato sensible

Ley 18.331, artículo 4:
Dato personal. Información de cualquier tipo referida a personas físicas o jurídicas determinadas o determinables.
Dato sensible. Datos personales que revelen origen racial y étnico, preferencias políticas, convicciones religiosas o morales, afiliación sindical e informaciones referentes a la salud o a la vida sexual.
Subir una foto a la API es enviarla a un tercero, que en el nivel gratuito la usa para mejorar sus productos.
En un ticket: nombre, cédula, dígitos de la tarjeta. Un ticket de farmacia puede revelar información de salud.



FARMACIA CENTRO
RUT 2187683280017
AV. ITALIA 1602 - MONTEVIDEO
24/09/2026 11:05

PARACETAMOL 500MG	187,00
ALCOHOL EN GEL 250ML	142,00
PROTECTOR SOLAR 50	890,00
CURITAS X20	98,00
SUBTOTAL	1.317,00
IVA INCLUIDO	237,49
TOTAL	1.317,00
TARJETA CREDITO ****0937	

GRACIAS POR SU COMPRA

Ley 18.331 (2008), artículo 4, texto publicado en IMPO. Uso de datos en el nivel gratuito: Google, «Pricing».

Para la bitácora

- Con un ticket propio: transcribirlo a mano y compararlo con lo que devolvió Gemini. Clasificar cada diferencia según la tabla de errores.
- ¿Cuántos tokens de entrada usó la solicitud (`promptTokenCount`)? ¿Cuántos corresponden a la imagen?
- ¿Cierra el control de consistencia? Si cierra, ¿alcanza para confiar en la tabla?
- ¿Hubo alguna alucinación? ¿Qué regla del prompt debió evitarla?
- ¿Qué datos personales tapaste antes de subir la foto?

Referencias

- Fielding, R., Nottingham, M. y Reschke, J. (2022). *HTTP Semantics*. RFC 9110. IETF.
- Gemini Team (2023). *Gemini: A Family of Highly Capable Multimodal Models*. arXiv:2312.11805.
- Gemma Team (2024). *Gemma: Open Models Based on Gemini Research and Technology*. arXiv:2403.08295.
- Gonzalez, R. C. y Woods, R. E. (2018). *Digital Image Processing* (4.ª ed.). Pearson.
- Goodfellow, I., Bengio, Y. y Courville, A. (2016). *Deep Learning*. MIT Press.
- Google. Documentación de la API de Gemini: «Tokens», «Image understanding», «generateContent», «Pricing», «Rate limits», «Using Gemini API keys», «Prompt design strategies». ai.google.dev (consultada el 29/09/2026).
- Ji, Z. et al. (2023). Survey of Hallucination in Natural Language Generation. *ACM Computing Surveys*, 55(12).
- Josefsson, S. (2006). *The Base16, Base32, and Base64 Data Encodings*. RFC 4648. IETF.
- Jurafsky, D. y Martin, J. H. *Speech and Language Processing* (3.ª ed., borrador en línea). web.stanford.edu/~jurafsky/slp3

- Kudo, T. y Richardson, J. (2018). SentencePiece: A simple and language independent subword tokenizer and detokenizer for Neural Text Processing. *EMNLP 2018: System Demonstrations*.
- Ley N.º 18.331 (2008). Protección de datos personales y acción de habeas data. IMPO.
- MDN Web Docs. An overview of HTTP. developer.mozilla.org
- Russell, S. y Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4.ª ed.). Pearson.
- Sennrich, R., Haddow, B. y Birch, A. (2016). Neural Machine Translation of Rare Words with Subword Units. *ACL 2016*.
- Vaswani, A. et al. (2017). Attention Is All You Need. *NeurIPS 2017*.
- Wickham, H. (2024). *httr2: Perform HTTP Requests and Process the Responses*. Paquete de R. httr2.r-lib.org



Speaker notes